Exercise 1: Process identification

Display all running processes with their PID, PPID, owner, CPU usage, memory usage, and command.

```
ps -eo pid,ppid,user,%cpu,%mem,cmd
```

List only the processes belonging to the current user.

```
ps -u $USER -o pid,ppid,user,%cpu,%mem,cmd
```

Identify the five processes consuming the most CPU.

```
ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%cpu | head -n 6
```

Identify the five processes consuming the most memory.

```
ps -eo pid,ppid,user,%cpu,%mem,cmd --sort=-%mem | head -n 6
```

Explain why high CPU usage alone does not confirm malicious activity.

```
High CPU usage alone indicates intensive computational workloads (e.g., video
rendering, code compilation, database indexing, or mathematical simulations).
Legitimate software frequently consumes full CPU capacity during normal operations.
Malicious activity is confirmed only by evaluating suspicious process paths,
unauthorized network connections, unverified binaries, or malicious persistence
mechanisms.
```

Exercise 2: Parent and child processes

Write a shell script that starts two background processes and displays their PIDs.

```
#!/bin/bash
sleep 300 &
PID1=$!
echo "Background Process 1 PID: $PID1"

sleep 300 &
PID2=$!
echo "Background Process 2 PID: $PID2"
```

Identify the PID and PPID of the shell script.

```
echo "Script PID: $$"
```

```
echo "Script PPID: $PPID"
```

Display the child processes created by the script.

```
pgrep -P $$ -l
```

Explain the purpose of $$, $!, &, and wait.

```
$$    : Evaluates to the Process ID (PID) of the currently running shell process.
$!    : Holds the Process ID (PID) of the most recently executed background job.
&     : Operator placed at the end of a command to run it asynchronously in the
background.
wait  : Pauses execution of the current script until specified background processes
exit.
```

Observe what happens to the child processes when the parent script terminates.

```
When the parent script terminates while child processes are still running, the child
processes become orphan processes. The system init daemon (PID 1, such as systemd)
automatically adopts these orphan processes and cleans them up upon completion.
```

Exercise 3: Process control using kill

Start a background process and record its PID.

```
sleep 600 &
PID=$!
echo "Target PID: $PID"
```

Suspend the process and verify its state.

```
kill -SIGSTOP $PID
ps -o pid,stat,cmd -p $PID
```

Resume the process and verify its state.

```
kill -SIGCONT $PID
ps -o pid,stat,cmd -p $PID
```

Terminate the process gracefully.

```
kill -SIGTERM $PID
```

Verify that the process has terminated.

```
ps -p $PID
```

Explain the differences between SIGSTOP, SIGCONT, SIGTERM, and SIGKILL.

```
SIGSTOP (19) : Pauses execution immediately. Cannot be caught, blocked, or ignored.
SIGCONT (18) : Resumes execution of a process previously paused by SIGSTOP.
SIGTERM (15) : Requests graceful termination, allowing the process to run cleanup
routines before exiting.
SIGKILL (9)  : Forces immediate kernel-level termination. Cannot be caught or ignored.
```

Exercise 4: Graceful and forced termination

Write a monitoring script that periodically records a timestamp in a log file.

```
#!/bin/bash
LOGFILE="monitor.log"
while true; do
    echo "$(date '+%Y-%m-%d %H:%M:%S') - Heartbeat" >> "$LOGFILE"
    sleep 2
done
```

Configure the script to record a shutdown message when it receives SIGTERM.

```
#!/bin/bash
LOGFILE="monitor.log"
cleanup() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - [SHUTDOWN] Received SIGTERM signal" >>
"$LOGFILE"
    exit 0
}
trap cleanup SIGTERM

while true; do
    echo "$(date '+%Y-%m-%d %H:%M:%S') - Heartbeat" >> "$LOGFILE"
    sleep 2
done
```

Terminate the script using SIGTERM and inspect the log.

```
./monitor.sh &
MON_PID=$!
sleep 4
kill -SIGTERM $MON_PID
cat monitor.log
```

Restart the script and terminate it using SIGKILL.

```
./monitor.sh &
MON_PID=$!
sleep 4
kill -SIGKILL $MON_PID
cat monitor.log
```

Compare the results and explain why they differ.

```
SIGTERM allows the script's signal trap handler to execute, cleanly writing the
'[SHUTDOWN] Received SIGTERM' log message before terminating. SIGKILL terminates the
process immediately at the OS kernel level, bypassing user-space code entirely, which
leaves no shutdown log.
```

Exercise 5: Multiple processes using killall

Start three instances of a lab program with a unique process name.

```
sleep 1000 & exec -a lab_app sleep 1000 &
sleep 1000 & exec -a lab_app sleep 1000 &
sleep 1000 & exec -a lab_app sleep 1000 &
```

Display the PID and owner of each instance.

```
pgrep -u $USER -l -a lab_app
```

Terminate one instance using its PID.

```
kill -9 <TARGET_PID>
```

Terminate the remaining instances using killall.

```
killall lab_app
```

Verify that all instances have terminated.

```
pgrep -l lab_app
```

Explain the risk of using killall with a common process name.

```
Using killall with common process names (e.g., killall python, killall bash) will
indiscriminately terminate all processes matching that name owned by the user. This
can crash critical background daemons, active interactive sessions, or unrelated
```

```
production services.
```

Exercise 6: Recurring process monitoring using cron

Write a shell script that records the current date, time, username, and number of processes owned by the current user.

```
#!/bin/bash
LOGFILE="/tmp/user_proc.log"
echo "$(date '+%Y-%m-%d %H:%M:%S') | User: $(whoami) | Processes: $(ps -u $(whoami) --
no-headers | wc -l)" >> "$LOGFILE"
```

Schedule the script to run every minute.

```
crontab -e
# Append line:
* * * * * /bin/bash /path/to/process_count.sh
```

Verify that the scheduled task executes successfully.

```
tail -f /tmp/user_proc.log
```

Start additional background processes and observe changes in the log.

```
sleep 300 & sleep 300 &
tail -f /tmp/user_proc.log
```

Modify the schedule to run every five minutes.

```
crontab -e
# Change schedule to:
*/5 * * * * /bin/bash /path/to/process_count.sh
```

Remove the exercise entry without deleting other cron entries.

```
crontab -e
# Delete only the line containing process_count.sh and save.
```

Exercise 7: Cron scheduling

Write cron entries for the following requirements:

Check failed-login logs every five minutes.

```
*/5 * * * * /usr/local/bin/check_failed_logins.sh
```

Capture a process snapshot at the beginning of every hour.

```
0 * * * * /usr/local/bin/proc_snapshot.sh
```

Back up security logs daily at 2:30 AM.

```
30 2 * * * /usr/local/bin/backup_sec_logs.sh
```

Generate a security report every weekday at 9:00 AM.

```
0 9 * * 1-5 /usr/local/bin/generate_sec_report.sh
```

Perform a file-integrity check every Sunday at 3:00 AM.

```
0 3 * * 0 /usr/local/bin/file_integrity_check.sh
```

Run a monitoring script every 15 minutes.

```
*/15 * * * * /usr/local/bin/monitor.sh
```

Generate a weekly report every Monday at 8:30 AM.

```
30 8 * * 1 /usr/local/bin/weekly_report.sh
```

Exercise 8: One-time scheduling using at

Schedule a task to record the current date and process list two minutes from now.

```
echo "date > /tmp/at_out.txt && ps aux >> /tmp/at_out.txt" | at now + 2 minutes
```

Display all pending jobs.

```
atq
```

Inspect the commands associated with the scheduled job.

```
at -c <JOB_ID>
```

Verify that the output file is created after execution.

```
cat /tmp/at_out.txt
```

Explain why the completed job no longer appears in the pending-job list.

```
The at daemon (atd) executes scheduled jobs exactly once. Upon execution, atd
automatically purges the job entry from the spool directory (/var/spool/at), so
finished tasks no longer remain in the pending queue (atq).
```

Exercise 9: Cancelling scheduled jobs

Schedule a harmless task to execute ten minutes from now.

```
echo "echo 'Test' > /tmp/test.txt" | at now + 10 minutes
```

Identify its job number.

```
atq
```

Cancel the scheduled task.

```
atrm <JOB_ID>
```

Verify that the task has been removed.

```
atq
```

Explain the difference between cancelling an at job and removing a cron entry.

```
Cancelling an at job (atrm) removes a single pending task queued for one-time
execution from /var/spool/at. Removing a cron entry (crontab -e) deletes a recurring
schedule rule from the user crontab file to halt future recurring executions.
```

Exercise 10: Incident-response scenario

An unfamiliar lab process has multiple running instances and a scheduled task that will restart it.

Identify all instances of the lab process and record their PIDs and owners.

```
ps -eo pid,ppid,user,%cpu,%mem,cmd | grep malicious_lab_proc | grep -v grep
```

Suspend one instance and verify its state.

```
kill -SIGSTOP 4521
ps -o pid,stat,cmd -p 4521
```

Resume the suspended instance.

```
kill -SIGCONT 4521
ps -o pid,stat,cmd -p 4521
```

Terminate all instances of the lab process.

```
killall -9 malicious_lab_proc
```

Identify and inspect the scheduled restart task.

```
crontab -l
ls -la /etc/cron.* /var/spool/cron/crontabs/
```

Cancel the scheduled restart.

```
crontab -e
# Delete the line restarting malicious_lab_proc
```

Verify that no instances or restart jobs remain.

```
pgrep -l malicious_lab_proc
crontab -l | grep malicious_lab_proc
```

Prepare an incident report documenting your observations, actions, and verification.

```
INCIDENT REPORT: LAB-PROC-UNAUTHORIZED-010
• Observations : Detected unauthorized process 'malicious_lab_proc' running under user
context with PIDs 4521 and 4522. Identified persistence cron job: '*/5 * * * *
/tmp/malicious_lab_proc'.
• Actions Taken: 1. Suspended PID 4521 via SIGSTOP for triage.
                 2. Resumed PID 4521 via SIGCONT.
                 3. Terminated all active instances via killall -9.
                 4. Removed restart cron entry via crontab -e.
• Verification : Confirmed 0 active processes via pgrep and 0 cron restart hooks via
crontab -l.
```